

Advanced Educational RAG System - Complete Implementation Timeline

Project: Production-Grade RAG with All Advanced Concepts

Duration: 6 Weeks

Stack: LangChain, LangGraph, Groq, Pinecone, BGE-M3, Redis, PostgreSQL

Status:  Not Started

Table of Contents

1. [Project Setup & Infrastructure \(Week 1\)](#)
 2. [Document Ingestion Pipeline \(Week 2\)](#)
 3. [Advanced Retrieval Implementation \(Week 3\)](#)
 4. [Generation & Agent System \(Week 4\)](#)
 5. [Evaluation & Guardrails \(Week 5\)](#)
 6. [Testing, Optimization & Deployment \(Week 6\)](#)
-

WEEK 1: Project Setup & Infrastructure

Goal: Get all infrastructure running and core framework ready

Status: ☐ Not Started

Day 1: Environment Setup & Dependencies

Time: 4-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **1.1** Create project directory structure

```
mkdir -p educational-rag/{config,src,data,tests,scripts,api,notebooks}
cd educational-rag
python -m venv venv
source venv/bin/activate
```

☐ **1.2** Create `requirements.txt` with all dependencies

- Core: langchain, langgraph, langchain-groq
- Embeddings: sentence-transformers, torch
- Vector DB: pinecone-client
- Database: psycopg2-binary, sqlalchemy, pgvector
- Cache: redis
- Document processing: pypdf, pdfplumber
- Evaluation: ragas
- API: fastapi, uvicorn
- Testing: pytest

☐ **1.3** Install all dependencies

```
pip install -r requirements.txt
```

☐ **1.4** Download BGE-M3 model (first time)

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('BAAI/bge-m3')
```

Acceptance Criteria:

- ☒ Virtual environment created and activated
- ☒ All packages installed without errors
- ☒ BGE-M3 model downloaded successfully
- ☒ `python -c "import langchain, langgraph, pinecone"` works

Day 2: Infrastructure Services Setup

Time: 4-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **2.1** Create `docker-compose.yml`
 - Redis service (port 6379)
 - PostgreSQL with pgvector (port 5432)
- ☐ **2.2** Start Docker services

```
docker-compose up -d
docker-compose ps # Verify all running
```

- ☐ **2.3** Test Redis connection

```
redis-cli ping # Should return PONG
```

- ☐ **2.4** Test PostgreSQL connection

```
psql postgresql://raguser:changeme@localhost:5432/educational_rag -c
"SELECT version();"

```

Acceptance Criteria:

- ☒ Docker services running
- ☒ Redis accessible on port 6379
- ☒ PostgreSQL accessible on port 5432
- ☒ pgvector extension installed

Day 3: External Services Configuration

Time: 3-4 hours

Status: ☐ Not Started

Tasks:

- ☐ **3.1** Create Groq account and get API key

- Visit: <https://console.groq.com>
- Create account
- Generate API key
- Test with sample request

☐ **3.2** Create Pinecone account and index

- Visit: <https://www.pinecone.io>
- Create free tier account
- Create index: `educational-rag`
- Dimension: 1024 (BGE-M3)
- Metric: cosine
- Cloud: AWS, Region: us-west-2

☐ **3.3** Create `.env` file with all credentials

```
GROQ_API_KEY=gsk_...
PINECONE_API_KEY=...
PINECONE_INDEX_NAME=educational-rag
DATABASE_URL=postgresql://raguser:changeme@localhost:5432/educational_rag
REDIS_URL=redis://localhost:6379
```

☐ **3.4** Create `.env.example` (without sensitive values)

Acceptance Criteria:

- ☒ Groq API key working (test with curl or Python)
- ☒ Pinecone index created and accessible
- ☒ `.env` file configured
- ☒ `.env.example` created for team

Day 4: Core Configuration & Database Schema

Time: 4-5 hours

Status: ☐ Not Started

Tasks:

☐ **4.1** Create `config/settings.py`

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    groq_api_key: str
    pinecone_api_key: str
    pinecone_index_name: str
    database_url: str
    redis_url: str
    # ... all other configs

    class Config:
        env_file = ".env"
```

☐ **4.2** Create `config/prompts.py` with all system prompts

- System prompt for generation
- Self-reflection prompts
- Query decomposition prompts
- HyDE generation prompt

☐ **4.3** Create `config/model_config.py`

- LLM parameters (temperature, max_tokens)
- Retrieval parameters (top_k values)
- Chunking parameters

☐ **4.4** Create database schema in `src/memory/database.py`

```
from sqlalchemy import Column, String, Text, DateTime, JSON
from pgvector.sqlalchemy import Vector
```

```
class Conversation(Base):
    __tablename__ = 'conversations'
    id = Column(String, primary_key=True)
    conversation_id = Column(String, index=True)
    user_id = Column(String, index=True)
    role = Column(String)
    message = Column(Text)
    sources_used = Column(JSON)
    metadata = Column(JSON)
    timestamp = Column(DateTime)
```

- ☐ **4.5** Create `scripts/init_database.py` to create tables
- ☐ **4.6** Run database initialization

```
python scripts/init_database.py
```

Acceptance Criteria:

- ☒ Settings module loads environment variables correctly
- ☒ All config files created
- ☒ Database tables created successfully
- ☒ Can insert and query from conversations table

Day 5: LangGraph State & Basic Workflow

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **5.1** Create `src/graph/state.py` with complete RAGState

```

from typing import TypedDict, List, Dict, Optional

class RAGState(TypedDict):
    # Input
    query: str
    user_id: str
    conversation_id: str
    # ... all 50+ state fields

```

- ☐ **5.2** Create basic node structure in `src/graph/nodes/`
 - `input_nodes.py`: `receive_query`, `validate_input`
 - `retrieval_nodes.py`: `embed_query`, `retrieve_candidates`
 - `generation_nodes.py`: `generate_response`
 - `utility_nodes.py`: `cache_check`, `log_metrics`
- ☐ **5.3** Create `src/graph/workflow.py` with basic workflow

```

from langgraph.graph import StateGraph, END

workflow = StateGraph(RAGState)
workflow.add_node("receive_query", receive_query)
workflow.add_node("embed_query", embed_query)
workflow.add_node("retrieve", retrieve_candidates)
workflow.add_node("generate", generate_response)

workflow.set_entry_point("receive_query")
workflow.add_edge("receive_query", "embed_query")
workflow.add_edge("embed_query", "retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

app = workflow.compile()

```

- ☐ **5.4** Create simple test to verify workflow runs

```
result = app.invoke({
    "query": "test query",
    "user_id": "test_user",
    "conversation_id": "test_conv"
})
```

Acceptance Criteria:

- ☒ RAGState defined with all fields
- ☒ Basic nodes created and importable
- ☒ Simple workflow compiles without errors
- ☒ Test workflow runs end-to-end (even with placeholder logic)



WEEK 2: Document Ingestion Pipeline

Goal: Build complete document processing and indexing pipeline

Status: ☐ Not Started

Day 6: PDF Processing

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **6.1** Create `src/ingestion/base_loader.py` with abstract loader class

```
from abc import ABC, abstractmethod

class BaseLoader(ABC):
    @abstractmethod
    def load(self, file_path: str) -> List[Dict]:
        pass
```

- ☐ **6.2** Create `src/ingestion/pdf_loader.py`
 - Use pypdf for text extraction
 - Preserve page numbers

- Extract metadata (title, author, subject)
 - Handle multi-column layouts
 - Detect and preserve formulas (LaTeX)
- ☐ **6.3** Create `src/ingestion/chunking_strategies.py`
- Semantic chunking by paragraphs
 - Fixed-size chunking with overlap (500-800 tokens, 100 overlap)
 - Preserve headings with chunks
 - Keep formulas intact
- ☐ **6.4** Test with sample PDFs

```
loader = PDFLoader("data/raw/pdfs/sample.pdf")
chunks = loader.load_and_chunk()
assert len(chunks) > 0
assert all('page_number' in chunk for chunk in chunks)
```

Acceptance Criteria:

- ☒ Can load and extract text from PDFs
- ☒ Page numbers preserved
- ☒ Chunks created with proper overlap
- ☒ Metadata extracted correctly
- ☒ Test passes with 3+ different PDFs

Day 7: JSON Transcript Processing

Time: 4-5 hours

Status: ☐ Not Started

Tasks:

- ☐ **7.1** Create `src/ingestion/transcript_loader.py`
- Parse JSON structure
 - Extract timestamps
 - Handle different JSON schemas

- Clean speaker labels
- ☐ **7.2** Create time-based chunking strategy
- Chunk by 2-3 minute segments
- Respect sentence boundaries
- Preserve timestamp metadata
- Format: "Speaker [00:05:23]: Text..."
- ☐ **7.3** Test with sample transcripts

```
loader = TranscriptLoader("data/raw/transcripts/sample.json")
chunks = loader.load_and_chunk()
assert all('timestamp' in chunk for chunk in chunks)
assert all('video_title' in chunk for chunk in chunks)
```

Acceptance Criteria:

- ☒ Can parse JSON transcripts
- ☒ Timestamps extracted and preserved
- ☒ Chunks respect time boundaries
- ☒ Test passes with 3+ different transcript formats

Day 8: Parent-Child Chunking Implementation

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **8.1** Create `src/ingestion/parent_child_chunker.py`
 - Parent: Large context (800-1000 tokens)
 - Child: Small precise chunks (200-300 tokens)
 - Maintain parent-child relationships
 - Store both in metadata
- ☐ **8.2** Update PDF loader to use parent-child
 - Parent: Full sections/chapters

- Child: Paragraphs within sections
- ☐ **8.3** Update transcript loader to use parent-child
- Parent: Full topic segment (5-10 minutes)
- Child: Individual statements (30-60 seconds)
- ☐ **8.4** Create test for hierarchy preservation

```
chunks = parent_child_chunker.chunk(document)
for child in chunks:
    assert 'parent_id' in child
    assert 'parent_text' in child
    assert len(child['text']) < len(child['parent_text'])
```

Acceptance Criteria:

- ☒ Parent-child relationships established
 - ☒ Both parent and child text stored
 - ☒ Can retrieve child, expand to parent
 - ☒ Test verifies hierarchy integrity
-

Day 9: Metadata Extraction & Enrichment

Time: 4-5 hours

Status: ☐ Not Started

Tasks:

- ☐ **9.1** Create `src/ingestion/metadata_extractor.py`
 - Extract topics using LLM
 - Classify difficulty level
 - Detect subject area
 - Extract key concepts
- ☐ **9.2** Implement topic extraction

```
def extract_topics(text: str) -> List[str]:
    prompt = f"Extract 3-5 main topics from: {text}"
    response = llm.invoke(prompt)
    return parse_topics(response)
```

☐ 9.3 Create metadata schema

```
chunk_metadata = {
    "source_type": "pdf" | "video",
    "source_name": str,
    "page_number": int | None,
    "timestamp": str | None,
    "topics": List[str],
    "difficulty": "beginner" | "intermediate" | "advanced",
    "subject": str,
    "chunk_id": str,
    "parent_id": str
}
```

☐ 9.4 Apply to all processed documents

Acceptance Criteria:

- ☒ Topics extracted automatically
- ☒ Difficulty classified correctly
- ☒ All metadata fields populated
- ☒ Metadata stored with each chunk

Day 10: Embedding Generation & Pinecone Indexing

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **10.1** Create `src/embeddings/embedding_model.py`

```

from sentence_transformers import SentenceTransformer

class BGEEEmbedder:
    def __init__(self):
        self.model = SentenceTransformer('BAAI/bge-m3')

    def embed(self, texts: List[str]) -> List[List[float]]:
        return self.model.encode(texts).tolist()

```

☐ **10.2** Create `src/embeddings/batch_embedder.py`

- Batch processing (32 chunks at a time)
- Progress tracking with tqdm
- Error handling and retry logic
- Cache embeddings locally

☐ **10.3** Create `src/vectorstore/pinecone_manager.py`

```

from pinecone import Pinecone

class PineconeManager:
    def upsert_chunks(self, chunks: List[Dict]):
        vectors = []
        for chunk in chunks:
            vectors.append({
                "id": chunk["chunk_id"],
                "values": chunk["embedding"],
                "metadata": chunk["metadata"]
            })
        self.index.upsert(vectors=vectors)

```

☐ **10.4** Create `scripts/ingest_documents.py`

- Load all PDFs and transcripts
- Process and chunk
- Generate embeddings
- Upload to Pinecone

- Log progress and stats

☐ 10.5 Run ingestion pipeline

```
python scripts/ingest_documents.py \  
  --pdf-dir data/raw/pdfs \  
  --transcript-dir data/raw/transcripts
```

Acceptance Criteria:

- ☒ Embeddings generated for all chunks
- ☒ All chunks uploaded to Pinecone
- ☒ Can query Pinecone and retrieve chunks
- ☒ Metadata preserved in Pinecone
- ☒ Parent-child relationships maintained

Verification:

```
# Test retrieval  
query = "photosynthesis"  
query_embedding = embedder.embed([query])[0]  
results = pinecone_manager.query(query_embedding, top_k=5)  
assert len(results) == 5  
assert all('metadata' in r for r in results)
```



WEEK 3: Advanced Retrieval Implementation

Goal: Implement all advanced retrieval strategies

Status: ☐ Not Started

Day 11: Basic Retrieval + Hybrid Search

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ 11.1 Create `src/retrieval/vector_retriever.py`

- Query Pinecone with embedding
 - Apply metadata filters
 - Return top-k results
- ☐ **11.2** Create `src/retrieval/hybrid_retriever.py`
- Implement BM25 keyword search
 - Reciprocal Rank Fusion (RRF) algorithm
 - Merge vector + keyword results
- ☐ **11.3** Test both retrievers

```
# Vector only
results_vector = vector_retriever.retrieve("photosynthesis", top_k=20)

# Hybrid
results_hybrid = hybrid_retriever.retrieve("photosynthesis", top_k=20)

# Verify hybrid includes both vector and keyword matches
```

Acceptance Criteria:

- ☒ Vector retrieval working
- ☒ BM25 keyword search working
- ☒ RRF fusion combines results correctly
- ☒ Hybrid retrieval returns diverse results

Day 12: Query Enhancement (Decomposition, Self-Querying)

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **12.1** Create `src/retrieval/query_decomposer.py`

```
def decompose_query(query: str) -> List[str]:
    prompt = f"""Break this complex query into simple sub-questions:
    Query: {query}

    Return 3-5 sub-questions that together answer the original."""

    sub_queries = llm.invoke(prompt)
    return parse_sub_queries(sub_queries)
```

☐ **12.2** Create `src/retrieval/self_query_retriever.py`

```
def extract_filters(query: str) -> Dict:
    prompt = f"""Extract structured filters from this query:
    Query: {query}
    Available filters: topic, source_type, difficulty, date_range

    Return JSON: {"{"topic": [...], "source_type": "...", ...}"}"""

    filters = llm.invoke(prompt)
    return parse_json(filters)
```

☐ **12.3** Create `src/retrieval/query_processor.py`

- Spell check and correction
- Query expansion (add synonyms)
- Incorporate chat history context

☐ **12.4** Test all query enhancements


```
# Decomposition
sub_queries = query_decomposer.decompose(
    "Compare C3 vs C4 photosynthesis in drought"
)
assert len(sub_queries) >= 3

# Self-querying
filters = self_query.extract_filters(
    "Show me advanced biology videos about photosynthesis"
)
assert filters["difficulty"] == "advanced"
assert filters["source_type"] == "video"
```

Acceptance Criteria:

- ☒ Query decomposition produces relevant sub-questions
 - ☒ Self-querying extracts filters accurately
 - ☒ Query expansion adds useful synonyms
 - ☒ All components integrate with retrieval pipeline
-

Day 13: HyDE & RAG Fusion

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **13.1** Create `src/embeddings/hyde_generator.py`

```
def generate_hypothetical_answer(query: str) -> str:
    prompt = f"""Write a hypothetical answer to this question.
    Don't worry about accuracy, just write plausible text that might
    appear in a document answering this question.

    Question: {query}

    Hypothetical Answer: """

    return llm.invoke(prompt)
```

☐ 13.2 Implement HyDE retrieval flow

- Generate hypothetical answer
- Embed the answer (not the query)
- Retrieve using answer embedding
- Often gets better semantic matches

☐ 13.3 Create `src/retrieval/fusion_retriever.py`

```
def fusion_retrieve(query: str, top_k: int) -> List[Dict]:
    # Generate 3-5 query variations
    variations = generate_variations(query)

    # Retrieve for each variation
    all_results = []
    for var in variations:
        results = retrieve(var, top_k=top_k)
        all_results.append(results)

    # Apply Reciprocal Rank Fusion
    fused = reciprocal_rank_fusion(all_results)
    return fused[:top_k]
```

☐ 13.4 Test HyDE and Fusion

```
# HyDE
hyde_results = hyde_retriever.retrieve("How does photosynthesis work?")

# Fusion
fusion_results = fusion_retriever.retrieve("photosynthesis mechanism")

# Compare with baseline
baseline_results = vector_retriever.retrieve("photosynthesis mechanism")

# Fusion should retrieve more diverse results
```

Acceptance Criteria:

- ☒ HyDE generates plausible hypothetical answers
 - ☒ HyDE retrieval finds relevant documents
 - ☒ RAG Fusion generates diverse query variations
 - ☒ Fusion results show better coverage than single query
-

Day 14: Ensemble Retrieval & Reranking

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **14.1** Create `src/retrieval/ensemble_retriever.py`

```

class EnsembleRetriever:
    def __init__(self):
        # Primary: BGE-M3
        self.bge_model = SentenceTransformer('BAAI/bge-m3')
        # Optional: Add more models if budget allows
        # self.openai_embedder = OpenAIEmbeddings()

    def retrieve(self, query: str, top_k: int):
        # Retrieve with each model
        bge_results = self.retrieve_with_bge(query, top_k)
        # Combine results with voting
        return self.vote(bge_results)

```

☐ 14.2 Create `src/retrieval/reranker.py`

```

from sentence_transformers import CrossEncoder

class CrossEncoderReranker:
    def __init__(self):
        self.model = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2')

    def rerank(self, query: str, chunks: List[Dict], top_k: int):
        # Score each (query, chunk) pair
        pairs = [(query, chunk['text']) for chunk in chunks]
        scores = self.model.predict(pairs)

        # Re-sort by cross-encoder scores
        for i, chunk in enumerate(chunks):
            chunk['rerank_score'] = scores[i]

        reranked = sorted(chunks, key=lambda x: x['rerank_score'],
                           reverse=True)
        return reranked[:top_k]

```

☐ 14.3 Integrate reranking into retrieval pipeline

- Retrieve 20-30 candidates with vector search
- Rerank to top 5-8 with cross-encoder

☐ 14.4 Test reranking quality

```
# Retrieve candidates
candidates = hybrid_retriever.retrieve(query, top_k=30)

# Rerank
reranked = reranker.rerank(query, candidates, top_k=5)

# Verify reranked scores > original scores
assert reranked[0]['rerank_score'] > reranked[-1]['rerank_score']
```

Acceptance Criteria:

- ☒ Ensemble retrieval combines multiple models
- ☒ Cross-encoder reranker implemented
- ☒ Reranking improves top-5 precision
- ☒ Performance acceptable (rerank 30 docs in <300ms)

Day 15: Adaptive Retrieval & Integration

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ 15.1 Create `src/retrieval/adaptive_retriever.py`

```
def determine_retrieval_depth(query: str, query_intent: str) -> int:
    """Dynamically decide top_k based on query complexity"""
    if query_intent == "simple_fact":
        return 3
    elif query_intent == "explanation":
        return 8
    elif query_intent == "comparison":
        return 15
    elif query_intent == "complex_multi_part":
        return 30
    else:
        return 10 # default
```

☐ **15.2** Integrate all retrieval strategies into workflow

- Create unified retrieval interface
- Strategy selection based on query
- Parallel execution of multiple strategies
- Result merging and deduplication

☐ **15.3** Create comprehensive retrieval test suite

```
# Test each strategy
test_queries = [
    "What is photosynthesis?", # Simple
    "Compare C3 and C4 photosynthesis", # Comparison
    "How do C3 plants adapt to drought stress and what is the water use efficiency calculation?" # Complex
]

for query in test_queries:
    results = unified_retriever.retrieve(query)
    assert len(results) > 0
    assert all('text' in r for r in results)
    assert all('metadata' in r for r in results)
```

Acceptance Criteria:

- ☒ Adaptive retrieval adjusts top_k dynamically
 - ☒ All retrieval strategies integrated
 - ☒ Unified interface for all strategies
 - ☒ Test suite passes for all query types
 - ☒ Can switch strategies based on configuration
-



WEEK 4: Generation & Agent System

Goal: Implement LLM generation, tool use, and agentic behavior

Status: ☐ Not Started

Day 16: LLM Integration & Prompt Engineering

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **16.1** Create `src/generation/llm_client.py`

```
from langchain_groq import ChatGroq

class GroqClient:
    def __init__(self):
        self.llm = ChatGroq(
            model="llama-3.1-70b-versatile",
            temperature=0.3,
            max_tokens=1500
        )

    def generate(self, prompt: str, stream: bool = False):
        if stream:
            return self.llm.stream(prompt)
        else:
            return self.llm.invoke(prompt)
```

- ☐ **16.2** Create `src/generation/prompt_builder.py`

```
def build_rag_prompt(query: str, context: str, chat_history: str) ->
str:
    return f"""You are an expert educational assistant.

    Context from sources:
    {context}

    Previous conversation:
    {chat_history}

    Student question: {query}

    Instructions:
    1. Answer using ONLY the provided context
    2. Cite every claim with [Source N]
    3. For videos, include timestamps
    4. Be clear and educational
    5. If context doesn't contain the answer, say so

    Answer: """
```

☐ 16.3 Create `src/generation/response_parser.py`

- Extract citations from response
- Format video timestamps as links
- Parse tool calls if present
- Validate citation format

☐ 16.4 Test generation pipeline

```
context = get_retrieved_context()
prompt = prompt_builder.build(query, context, history)
response = llm_client.generate(prompt)
parsed = response_parser.parse(response)

assert 'answer' in parsed
assert 'citations' in parsed
```


Acceptance Criteria:

- ☒ LLM client connects to Groq
 - ☒ Prompts generate coherent responses
 - ☒ Citations extracted correctly
 - ☒ Response parsing handles all formats
-

Day 17: Context Construction & Management

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **17.1** Create context formatting functions

```
def format_context_with_sources(chunks: List[Dict]) -> str:
    formatted = "=== CONTEXT FROM SOURCES ===\n\n"

    for i, chunk in enumerate(chunks, 1):
        meta = chunk['metadata']
        source_info = format_source_info(meta)

        formatted += f"[Source {i} - {source_info}]\n"
        formatted += f"{chunk['text']}\n\n"

    return formatted
```

- ☐ **17.2** Implement parent-child context expansion
 - Retrieve child chunks (small, precise)
 - Expand to parent context (larger, more complete)
 - Avoid duplicates if multiple children share parent
- ☐ **17.3** Create diversity checker

```
def check_context_diversity(chunks: List[Dict]) -> float:
    """Ensure context covers different aspects"""
    topics = [c['metadata']['topics'] for c in chunks]
    unique_topics = set(t for topics_list in topics for t in
topics_list)

    source_types = [c['metadata']['source_type'] for c in chunks]
    unique_sources = set(source_types)

    diversity_score = (len(unique_topics) / 10) * 0.7 + \
        (len(unique_sources) / 2) * 0.3
    return min(diversity_score, 1.0)
```

☐ 17.4 Test context construction

```
retrieved = retriever.retrieve(query, top_k=15)
expanded = parent_child_expander.expand(retrieved)
context = format_context(expanded)
diversity = check_diversity(expanded)

assert diversity > 0.6 # Good coverage
assert len(context) < 8000 # Within token limits
```

Acceptance Criteria:

- ☒ Context formatted with clear source attribution
- ☒ Parent-child expansion working
- ☒ Diversity checker prevents redundancy
- ☒ Context fits within token limits

Day 18: Tool Integration (Calculator, Code Execution)

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

☐ **18.1** Create `src/tools/calculator.py`

```
import ast
import operator

class SafeCalculator:
    OPERATORS = {
        ast.Add: operator.add,
        ast.Sub: operator.sub,
        ast.Mult: operator.mul,
        ast.Div: operator.truediv,
        ast.Pow: operator.pow
    }

    def calculate(self, expression: str) -> float:
        """Safely evaluate mathematical expressions"""
        tree = ast.parse(expression, mode='eval')
        return self._eval(tree.body)
```

☐ **18.2** Create `src/tools/code_executor.py` (optional, for advanced)

- Sandbox Python code execution
- Security restrictions
- Timeout handling

☐ **18.3** Create `src/tools/tool_registry.py`

```
AVAILABLE_TOOLS = {
    "calculator": {
        "function": SafeCalculator().calculate,
        "description": "Perform mathematical calculations",
        "parameters": {"expression": "string"}
    }
}
```

☐ **18.4** Create `src/agents/tool_agent.py`

```
def detect_tool_need(query: str) -> List[str]:
    """Detect if query needs tools"""
    if any(word in query.lower() for word in ['calculate', 'compute',
'difference']):
        return ['calculator']
    return []

def execute_tool_call(tool_name: str, tool_input: str):
    tool = AVAILABLE_TOOLS[tool_name]
    return tool['function'](tool_input)
```

☐ 18.5 Test tool integration

```
query = "Calculate the water use efficiency difference between C3 and
C4"
tools_needed = detect_tool_need(query)
assert 'calculator' in tools_needed

result = execute_tool_call('calculator', '3.5 / 2.0')
assert result == 1.75
```

Acceptance Criteria:

- ☒ Calculator tool works safely
- ☒ Tool detection identifies when tools needed
- ☒ Tool execution returns correct results
- ☒ Error handling for invalid inputs

Day 19: Self-Reflection & Corrective RAG

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ 19.1 Create `src/generation/self_reflection.py`

```

def self_reflect(query: str, answer: str, context: str) -> Dict:
    reflection_prompt = f"""Review this Q&A:

    Question: {query}
    Your Answer: {answer}
    Context Used: {context}

    Rate your confidence (0-1) on:
    1. Accuracy of information
    2. Completeness of answer
    3. Quality of citations

    Are you uncertain about anything?
    Should you retrieve more information?

    Return JSON:
    {{
        "accuracy_confidence": 0.0-1.0,
        "completeness_confidence": 0.0-1.0,
        "citation_confidence": 0.0-1.0,
        "uncertainties": [...],
        "needs_more_retrieval": true/false
    }}"""

    return llm.invoke(reflection_prompt)

```

☐ **19.2** Create `src/agents/corrective_agent.py`

```
def assess_retrieval_quality(chunks: List[Dict], confidence: float) -> str:
    """Assess if retrieved docs are good enough"""
    if confidence < 0.4:
        return "poor"
    elif confidence < 0.7:
        return "uncertain"
    else:
        return "good"

def corrective_action(assessment: str, query: str):
    """Take corrective action if retrieval poor"""
    if assessment == "poor":
        # Expand search
        new_chunks = retriever.retrieve(query, top_k=50)
        # Try alternative phrasings
        alternative = rephrase_query(query)
        more_chunks = retriever.retrieve(alternative, top_k=30)
        return new_chunks + more_chunks
    return []
```

☐ 19.3 Integrate into workflow

- After first generation, trigger self-reflection
- If confidence low, trigger corrective RAG
- Retrieve more, regenerate answer
- Maximum 2 iterations

☐ 19.4 Test self-reflection

```
# Test with ambiguous query
query = "unclear vague question"
answer = generate(query, poor_context)
reflection = self_reflect(query, answer, poor_context)

assert reflection['needs_more_retrieval'] == True
```

Acceptance Criteria:

- ☒ Self-reflection assesses answer quality
 - ☒ Corrective RAG triggers on low confidence
 - ☒ Retrieval expands when needed
 - ☒ System doesn't loop infinitely (max 2 iterations)
-

Day 20: Multi-Hop Reasoning (Optional Advanced)

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **20.1** Create `src/agents/multi_hop_agent.py`

```
def detect_multi_hop(query: str) -> bool:
    """Detect if query requires multiple retrieval steps"""
    multi_hop_patterns = [
        "who taught the person who",
        "what influenced the work that",
        "where did the author of"
    ]
    return any(pattern in query.lower() for pattern in
multi_hop_patterns)

def execute_multi_hop(query: str) -> str:
    # Step 1: Answer intermediate question
    intermediate_query = extract_intermediate(query)
    intermediate_answer = rag_pipeline(intermediate_query)

    # Step 2: Use intermediate answer in follow-up
    final_query = construct_final_query(query, intermediate_answer)
    final_answer = rag_pipeline(final_query)

    return final_answer
```

- ☐ **20.2** Test multi-hop reasoning

```
query = "Who taught the scientist who discovered the Calvin cycle?"
is_multi_hop = detect_multi_hop(query)
assert is_multi_hop == True

answer = execute_multi_hop(query)
assert "gilbert" in answer.lower() # Gilbert N. Lewis
```

Acceptance Criteria:

- ☒ Multi-hop queries detected
 - ☒ Intermediate steps executed
 - ☒ Final answer synthesizes all steps
 - ☒ Test passes for 2-3 multi-hop queries
-



WEEK 5: Evaluation & Guardrails

Goal: Implement quality assurance, safety, and evaluation

Status: ☐ Not Started

Day 21: Input Validation & Guardrails

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **21.1** Create `src/guardrails/input_validator.py`


```

class InputValidator:
    def validate(self, query: str, user_id: str) -> Dict:
        checks = {
            "length": self.check_length(query),
            "content_safety": self.check_content(query),
            "rate_limit": self.check_rate_limit(user_id),
            "prompt_injection": self.detect_injection(query)
        }

        passed = all(checks.values())
        return {"passed": passed, "checks": checks}

    def check_length(self, query: str) -> bool:
        return 3 <= len(query) <= 500

    def check_content(self, query: str) -> bool:
        # Check for inappropriate content
        banned_words = [...]
        return not any(word in query.lower() for word in banned_words)

```

❑ **21.2** Create `src/guardrails/academic_integrity.py`

```
def detect_homework_cheating(query: str) -> bool:
    """Detect if student asking for homework answers"""
    cheating_patterns = [
        r"solve this problem for me",
        r"what's the answer to question \d+",
        r"give me the solution to"
    ]
    return any(re.search(pattern, query.lower()) for pattern in
cheating_patterns)

def educational_response_filter(query: str, response: str) -> str:
    """Modify response to guide rather than solve"""
    if detect_homework_cheating(query):
        return f"I'll guide you to solve this
yourself:\n\n{response}\n\nNow try applying these concepts to your
problem!"
    return response
```

☐ 21.3 Test input validation

```
# Valid query
result = validator.validate("How does photosynthesis work?", "user123")
assert result['passed'] == True

# Too short
result = validator.validate("hi", "user123")
assert result['passed'] == False
assert result['checks']['length'] == False
```

Acceptance Criteria:

- ☒ Length validation working
- ☒ Content safety checks implemented
- ☒ Rate limiting functional
- ☒ Academic integrity detection working
- ☒ All test cases pass

Day 22: Output Validation & Hallucination Detection

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **22.1** Create `src/guardrails/hallucination_detector.py`

```
def detect_hallucinations(response: str, context: str) -> float:
    """Score how much response is supported by context"""

    # Extract claims from response
    claims = extract_claims(response)

    verified_claims = 0
    for claim in claims:
        if verify_claim_in_context(claim, context):
            verified_claims += 1

    hallucination_score = 1.0 - (verified_claims / len(claims))
    return hallucination_score

def verify_claim_in_context(claim: str, context: str) -> bool:
    """Check if claim is supported by context"""
    # Use semantic similarity
    claim_embedding = embedder.embed([claim])[0]
    context_sentences = split_sentences(context)
    context_embeddings = embedder.embed(context_sentences)

    similarities = cosine_similarity([claim_embedding],
    context_embeddings)
    max_similarity = np.max(similarities)

    return max_similarity > 0.75 # Threshold
```

- ☐ **22.2** Create citation validator

```
def validate_citations(response: str, sources: List[Dict]) -> bool:
    """Verify all citations reference actual sources"""
    citation_pattern = r'\[Source (\d+)\]'
    cited_sources = re.findall(citation_pattern, response)

    for source_num in cited_sources:
        if int(source_num) > len(sources):
            return False # Invalid citation

    return True
```

☐ 22.3 Create `src/guardrails/output_validator.py`

```
class OutputValidator:
    def validate(self, response: str, context: str, sources: List[Dict])
    -> Dict:
        return {
            "hallucination_score": detect_hallucinations(response,
context),
            "citations_valid": validate_citations(response, sources),
            "content_safe": check_content_safety(response),
            "educational_tone": check_educational_tone(response)
        }
```

☐ 22.4 Test output validation

```
# Good response
response = "C3 plants use the Calvin cycle [Source 1]..."
validation = validator.validate(response, context, sources)
assert validation['hallucination_score'] < 0.2
assert validation['citations_valid'] == True

# Hallucinated response
fake_response = "Plants can photosynthesize on Mars [Source 1]"
validation = validator.validate(fake_response, context, sources)
assert validation['hallucination_score'] > 0.5
```

Acceptance Criteria:

- ☒ Hallucination detection working
 - ☒ Citation validation functional
 - ☒ Content safety checks implemented
 - ☒ False positives < 10%
 - ☒ Test suite passes
-

Day 23: RAGAS Evaluation Setup

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **23.1** Create test dataset in `data/evaluation/test_queries.json`

```
[
  {
    "query": "What is photosynthesis?",
    "ground_truth": "Photosynthesis is the process by which
plants...",
    "expected_sources": ["Biology Textbook Chapter 8"],
    "difficulty": "easy"
  },
  {
    "query": "Compare C3 and C4 photosynthesis",
    "ground_truth": "C3 and C4 plants differ in...",
    "expected_sources": ["Biology Textbook", "Plant Physiology
Video"],
    "difficulty": "medium"
  }
]
```

- ☐ **23.2** Create `src/evaluation/ragas_evaluator.py`

```

from ragas import evaluate
from ragas.metrics import (
    context_precision,
    context_recall,
    faithfulness,
    answer_relevancy
)

class RAGASEvaluator:
    def __init__(self):
        self.metrics = [
            context_precision,
            context_recall,
            faithfulness,
            answer_relevancy
        ]

    def evaluate_query(self, query: str, answer: str,
                      contexts: List[str], ground_truth: str) -> Dict:
        data = {
            "question": [query],
            "answer": [answer],
            "contexts": [contexts],
            "ground_truth": [ground_truth]
        }

        result = evaluate(data, metrics=self.metrics)
        return result

```

☐ **23.3** Create `src/evaluation/custom_metrics.py`

```
def calculate_citation_accuracy(response: str, sources: List[Dict]) -> float:
    """Custom metric: Are citations accurate?"""
    pass

def calculate_educational_quality(response: str) -> float:
    """Custom metric: Is response educational?"""
    pass

def calculate_response_completeness(response: str, query: str) -> float:
    """Custom metric: Does response fully address query?"""
    pass
```

☐ 23.4 Create evaluation pipeline

```
# scripts/run_evaluation.py
def run_full_evaluation():
    test_queries = load_test_queries()
    results = []

    for test in test_queries:
        # Run RAG pipeline
        response = rag_system.query(test['query'])

        # Evaluate with RAGAS
        ragas_scores = ragas_evaluator.evaluate(
            test['query'],
            response['answer'],
            response['contexts'],
            test['ground_truth']
        )

        # Custom metrics
        custom_scores = {
            'citation_accuracy': calculate_citation_accuracy(...),
            'educational_quality': calculate_educational_quality(...)
        }

        results.append(**ragas_scores, **custom_scores)

    return aggregate_results(results)
```

☐ 23.5 Run initial evaluation

```
python scripts/run_evaluation.py --output results/baseline_eval.json
```

Acceptance Criteria:

- ☒ Test dataset created with 20+ queries
- ☒ RAGAS metrics configured
- ☒ Custom metrics implemented
- ☒ Evaluation pipeline runs successfully

- ☒ Baseline scores recorded
-

Day 24: Chat History & Memory Integration

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

- ☐ **24.1** Create `src/memory/chat_history.py`

```

class ChatHistoryManager:
    def save_message(self, conversation_id: str, user_id: str,
                     role: str, message: str, metadata: Dict):
        """Save message to PostgreSQL"""
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO conversations
            (id, conversation_id, user_id, role, message, sources_used,
metadata, timestamp)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (generate_id(), conversation_id, user_id, role, message,
                json.dumps(metadata.get('sources', [])),
                json.dumps(metadata),
                datetime.now()))
        conn.commit()

    def get_history(self, conversation_id: str, limit: int = 10) ->
List[Dict]:
        """Retrieve last N messages"""
        cursor.execute("""
            SELECT role, message, timestamp, metadata
            FROM conversations
            WHERE conversation_id = %s
            ORDER BY timestamp DESC
            LIMIT %s
            """, (conversation_id, limit))
        return cursor.fetchall()

```

☐ **24.2** Create `src/memory/conversation_buffer.py`

```

class ConversationBuffer:
    def __init__(self, max_messages: int = 10):
        self.buffer = {}
        self.max_messages = max_messages

    def add_message(self, conversation_id: str, message: Dict):
        if conversation_id not in self.buffer:
            self.buffer[conversation_id] = []

        self.buffer[conversation_id].append(message)

        # Keep only last N messages
        if len(self.buffer[conversation_id]) > self.max_messages:
            self.buffer[conversation_id] = self.buffer[conversation_id]
            [-self.max_messages:]

    def get_context(self, conversation_id: str) -> str:
        """Format chat history for context"""
        messages = self.buffer.get(conversation_id, [])
        formatted = "Previous conversation:\n"
        for msg in messages:
            formatted += f"{msg['role']}: {msg['content']}\n"
        return formatted

```

☐ 24.3 Create `src/memory/long_term_memory.py`

```

class LongTermMemory:
    """Store user preferences and patterns"""

    def update_user_interests(self, user_id: str, topics: List[str]):
        """Track topics user asks about"""
        pass

    def get_user_level(self, user_id: str) -> str:
        """Infer user's knowledge level from history"""
        pass

```

☐ 24.4 Integrate into workflow

- Load history at start of query
- Include in context for generation
- Save response after generation

☐ 24.5 Test chat history

```
# Multi-turn conversation
conv_id = "test_conv_123"

# Turn 1
response1 = rag_system.query("What is photosynthesis?",
                             conversation_id=conv_id)

# Turn 2 (follow-up)
response2 = rag_system.query("Can you explain the light reactions in
                             more detail?", conversation_id=conv_id)

# Verify response2 references previous context
assert "as we discussed" in response2['answer'].lower() or "previously"
in response2['answer'].lower()
```

Acceptance Criteria:

- ☒ Chat history saved to PostgreSQL
- ☒ History loaded and formatted for context
- ☒ Follow-up questions understand context
- ☒ Conversation buffer manages memory
- ☒ Multi-turn test passes

Day 25: Caching Implementation

Time: 5-6 hours

Status: ☐ Not Started

Tasks:

☐ **25.1** Create `src/caching/redis_cache.py`

```
import redis
import json

class RedisCache:
    def __init__(self):
        self.client = redis.Redis(
            host=os.getenv('REDIS_HOST'),
            port=int(os.getenv('REDIS_PORT')),
            db=0,
            decode_responses=True
        )

    def set(self, key: str, value: any, ttl: int = 86400):
        """Store with TTL in seconds"""
        self.client.setex(key, ttl, json.dumps(value))

    def get(self, key: str) -> any:
        """Retrieve from cache"""
        value = self.client.get(key)
        return json.loads(value) if value else None
```

☐ **25.2** Create `src/caching/semantic_cache.py`

```

class SemanticCache:
    """Cache queries by semantic similarity"""

    def __init__(self):
        self.redis = RedisCache()
        self.embedder = BGEEEmbedder()

    def get_similar(self, query: str, threshold: float = 0.95) ->
Optional[Dict]:
        """Find cached response for similar query"""
        query_embedding = self.embedder.embed([query])[0]

        # Get all cached query embeddings
        cached_keys = self.redis.client.keys("query:*")

        for key in cached_keys:
            cached_data = self.redis.get(key)
            cached_embedding = cached_data['embedding']

            similarity = cosine_similarity([query_embedding],
[cached_embedding])[0][0]

            if similarity > threshold:
                return cached_data['response']

        return None

    def store(self, query: str, response: Dict):
        """Store query and response"""
        query_embedding = self.embedder.embed([query])[0]
        cache_data = {
            'query': query,
            'embedding': query_embedding,
            'response': response,
            'timestamp': datetime.now().isoformat()
        }

```

```
key = f"query:{hashlib.md5(query.encode()).hexdigest()}"
self.redis.set(key, cache_data, ttl=86400)
```

☐ **25.3** Create `src/caching/embedding_cache.py`

```
class EmbeddingCache:
    """Cache embeddings to avoid recomputation"""

    def get_or_embed(self, text: str) -> List[float]:
        cache_key = f"embed:{hashlib.md5(text.encode()).hexdigest()}"

        cached = self.redis.get(cache_key)
        if cached:
            return cached

        embedding = self.embedder.embed([text])[0]
        self.redis.set(cache_key, embedding, ttl=7*86400) # 7 days
        return embedding
```

☐ **25.4** Integrate caching into workflow

- Check semantic cache at start
- Cache embeddings during retrieval
- Cache final responses

☐ **25.5** Test caching

```
# First query
start = time.time()
response1 = rag_system.query("What is photosynthesis?")
time1 = time.time() - start

# Same query (should hit cache)
start = time.time()
response2 = rag_system.query("What is photosynthesis?")
time2 = time.time() - start

assert time2 < time1 * 0.3 # Cache should be 3x+ faster
assert response1['answer'] == response2['answer']
```

Acceptance Criteria:

- ☒ Redis cache working
- ☒ Semantic cache finds similar queries
- ☒ Embedding cache reduces computation
- ☒ Cache hits are 3x+ faster
- ☒ TTL management working



WEEK 6: Testing, Optimization & Deployment

Goal: Comprehensive testing, performance optimization, and deployment

Status: ☐ Not Started

Day 26: Complete LangGraph Workflow Integration

Time: 7-8 hours

Status: ☐ Not Started

Tasks:

- ☐ **26.1** Build complete workflow in `src/graph/workflow.py`


```
from langgraph.graph import StateGraph, END

def create_advanced_rag_workflow():
    workflow = StateGraph(RAGState)

    # Input phase
    workflow.add_node("receive_query", receive_query_node)
    workflow.add_node("check_cache", cache_lookup_node)
    workflow.add_node("validate_input", input_validation_node)
    workflow.add_node("classify_query", query_classification_node)

    # Query processing
    workflow.add_node("decompose_query", query_decomposition_node)
    workflow.add_node("self_query", self_querying_node)
    workflow.add_node("adaptive_plan", adaptive_retrieval_planning_node)

    # Retrieval phase
    workflow.add_node("hyde_retrieval", hyde_retrieval_node)
    workflow.add_node("fusion_retrieval", fusion_retrieval_node)
    workflow.add_node("ensemble_retrieval", ensemble_retrieval_node)
    workflow.add_node("merge_results", merge_retrieval_results_node)
    workflow.add_node("rerank", reranking_node)
    workflow.add_node("expand_context", parent_child_expansion_node)

    # Generation phase
    workflow.add_node("build_context", context_building_node)
    workflow.add_node("generate", generation_node)
    workflow.add_node("tool_use", tool_execution_node)

    # Validation phase
    workflow.add_node("self_reflect", self_reflection_node)
    workflow.add_node("validate_output", output_validation_node)
    workflow.add_node("corrective_rag", corrective_rag_node)

    # Finalization
    workflow.add_node("format_response", response_formatting_node)
    workflow.add_node("cache_response", response_caching_node)
```

```
workflow.add_node("save_history", history_saving_node)
workflow.add_node("log_metrics", metrics_logging_node)

# Define edges
workflow.set_entry_point("receive_query")
workflow.add_edge("receive_query", "check_cache")

# Conditional: cache hit -> return, miss -> continue
workflow.add_conditional_edges(
    "check_cache",
    lambda state: "return" if state['cache_hit'] else "validate",
    {
        "return": "format_response",
        "validate": "validate_input"
    }
)

workflow.add_edge("validate_input", "classify_query")
workflow.add_edge("classify_query", "decompose_query")
workflow.add_edge("decompose_query", "self_query")
workflow.add_edge("self_query", "adaptive_plan")

# Parallel retrieval strategies
workflow.add_edge("adaptive_plan", "hyde_retrieval")
workflow.add_edge("adaptive_plan", "fusion_retrieval")
workflow.add_edge("adaptive_plan", "ensemble_retrieval")

# Merge after parallel execution
workflow.add_edge(["hyde_retrieval", "fusion_retrieval",
"ensemble_retrieval"], "merge_results")
workflow.add_edge("merge_results", "rerank")

# Conditional: good confidence -> generate, low -> corrective
workflow.add_conditional_edges(
    "rerank",
    lambda state: "corrective" if state['retrieval_confidence'] <
0.6 else "expand",
```

```
        {
            "corrective": "corrective_rag",
            "expand": "expand_context"
        }
    )

    workflow.add_edge("corrective_rag", "expand_context")
    workflow.add_edge("expand_context", "build_context")
    workflow.add_edge("build_context", "generate")

    # Conditional: needs tools -> tool_use, else -> reflect
    workflow.add_conditional_edges(
        "generate",
        lambda state: "tools" if state['requires_tools'] else "reflect",
        {
            "tools": "tool_use",
            "reflect": "self_reflect"
        }
    )

    workflow.add_edge("tool_use", "self_reflect")

    # Conditional: needs regeneration -> back to generate, else ->
    validate
    workflow.add_conditional_edges(
        "self_reflect",
        lambda state: "regenerate" if state['needs_regeneration'] else
"validate",
        {
            "regenerate": "build_context",
            "validate": "validate_output"
        }
    )

    workflow.add_edge("validate_output", "format_response")
    workflow.add_edge("format_response", "cache_response")
    workflow.add_edge("cache_response", "save_history")
```

```
workflow.add_edge("save_history", "log_metrics")
workflow.add_edge("log_metrics", END)

return workflow.compile()
```

☐ 26.2 Test complete workflow

```
app = create_advanced_rag_workflow()

result = app.invoke({
    "query": "Compare C3 and C4 photosynthesis in drought conditions",
    "user_id": "test_user",
    "conversation_id": "test_conv"
})

assert 'response' in result
assert result['passed_output_guardrails'] == True
assert result['retrieval_confidence'] > 0.7
```

☐ 26.3 Debug and fix any issues

Acceptance Criteria:

- ☒ Complete workflow compiles
- ☒ All nodes execute correctly
- ☒ Conditional routing works
- ☒ End-to-end test passes
- ☒ No errors in workflow execution

Day 27: Unit Testing & Integration Testing

Time: 7-8 hours

Status: ☐ Not Started

Tasks:

☐ 27.1 Create unit tests for all components

```
# tests/test_ingestion.py
def test_pdf_loader():
    loader = PDFLoader("test.pdf")
    chunks = loader.load_and_chunk()
    assert len(chunks) > 0
    assert all('page_number' in c for c in chunks)

# tests/test_retrieval.py
def test_vector_retrieval():
    results = vector_retriever.retrieve("test query", top_k=5)
    assert len(results) == 5

def test_hybrid_retrieval():
    results = hybrid_retriever.retrieve("test query", top_k=10)
    assert len(results) <= 10

def test_reranking():
    candidates = [...]
    reranked = reranker.rerank("query", candidates, top_k=3)
    assert reranked[0]['rerank_score'] > reranked[-1]['rerank_score']

# tests/test_generation.py
def test_llm_generation():
    response = llm_client.generate("test prompt")
    assert len(response) > 0

def test_citation_extraction():
    response = "Text with [Source 1] citation"
    citations = extract_citations(response)
    assert len(citations) == 1

# tests/test_guardrails.py
def test_input_validation():
    result = validator.validate("valid query", "user123")
    assert result['passed'] == True

def test_hallucination_detection():
```

```
score = detect_hallucinations(fake_response, context)
assert score > 0.5
```

```
# tests/test_caching.py
def test_semantic_cache():
    cache.store("query1", response1)
    retrieved = cache.get_similar("query1")
    assert retrieved is not None
```

☐ 27.2 Create integration tests

```
# tests/test_workflow.py
def test_complete_workflow():
    result = rag_system.query("What is photosynthesis?")
    assert 'answer' in result
    assert 'sources' in result
    assert 'confidence' in result

def test_multi_turn_conversation():
    conv_id = generate_id()
    r1 = rag_system.query("What is photosynthesis?",
conversation_id=conv_id)
    r2 = rag_system.query("How does it work in C4 plants?",
conversation_id=conv_id)
    # Verify r2 understands context from r1

def test_tool_usage():
    result = rag_system.query("Calculate 3.5 / 2.0")
    assert 'calculator' in result['metadata']['tools_used']
    assert '1.75' in result['answer']
```

☐ 27.3 Run all tests

```
pytest tests/ -v --cov=src --cov-report=html
```

☐ 27.4 Achieve >80% code coverage

Acceptance Criteria:

- ☒ All unit tests pass
 - ☒ All integration tests pass
 - ☒ Code coverage > 80%
 - ☒ No critical bugs found
-

Day 28: Performance Optimization

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **28.1** Profile the system

```
import cProfile
import pstats

profiler = cProfile.Profile()
profiler.enable()

result = rag_system.query("test query")

profiler.disable()
stats = pstats.Stats(profiler)
stats.sort_stats('cumulative')
stats.print_stats(20)
```

- ☐ **28.2** Identify bottlenecks
 - Measure time for each workflow node
 - Identify slowest operations
 - Check database query performance
- ☐ **28.3** Implement optimizations
 - Parallel retrieval strategies
 - Batch embedding generation

- Database query optimization (indexes)
- Redis connection pooling
- LLM request batching where possible

☐ **28.4** Optimize chunking

- Test different chunk sizes
- Measure impact on retrieval quality
- Find optimal chunk size/overlap

☐ **28.5** Optimize retrieval parameters

- Test different top_k values
- Measure precision/recall trade-offs
- Find optimal reranking threshold

☐ **28.6** Benchmark improvements

```
# Before optimization
baseline_latency = measure_average_latency(100_queries)

# After optimization
optimized_latency = measure_average_latency(100_queries)

improvement = (baseline_latency - optimized_latency) / baseline_latency
assert improvement > 0.2 # 20% improvement
```

Acceptance Criteria:

- ☒ Bottlenecks identified
- ☒ Average latency < 3 seconds
- ☒ 20%+ performance improvement
- ☒ Cache hit rate > 30% for repeated queries
- ☒ Benchmark results documented

Day 29: API Development & Documentation

Time: 6-7 hours

Status: ☐ Not Started

Tasks:

- ☐ **29.1** Create FastAPI application in `api/main.py`

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Educational RAG API", version="1.0.0")

class QueryRequest(BaseModel):
    query: str
    user_id: str
    conversation_id: Optional[str] = None

class QueryResponse(BaseModel):
    answer: str
    sources: List[Dict]
    confidence: Dict
    conversation_id: str
    metadata: Dict

@app.post("/query", response_model=QueryResponse)
async def query_endpoint(request: QueryRequest):
    try:
        result = rag_system.query(
            query=request.query,
            user_id=request.user_id,
            conversation_id=request.conversation_id
        )
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/feedback")
async def feedback_endpoint(feedback: FeedbackRequest):
    # Store user feedback
    pass

@app.get("/health")
```

```
async def health_check():  
    return {"status": "healthy"}
```

☐ 29.2 Add authentication (JWT tokens)

```
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials  
  
security = HTTPBearer()  
  
@app.post("/query")  
async def query_endpoint(  
    request: QueryRequest,  
    credentials: HTTPAuthorizationCredentials = Depends(security)  
):  
    user = verify_token(credentials.credentials)  
    # ... rest of endpoint
```

☐ 29.3 Add rate limiting

```
from slowapi import Limiter  
from slowapi.util import get_remote_address  
  
limiter = Limiter(key_func=get_remote_address)  
  
@app.post("/query")  
@limiter.limit("100/hour")  
async def query_endpoint(request: Request, ...):  
    pass
```

☐ 29.4 Create API documentation

- Swagger UI (automatic with FastAPI)
- README with examples
- Postman collection

☐ 29.5 Test API

```
# Start server
uvicorn api.main:app --reload --port 8000

# Test with curl
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "What is photosynthesis?", "user_id": "test_user"}'
```

Acceptance Criteria:

- ☒ API endpoints working
 - ☒ Authentication implemented
 - ☒ Rate limiting functional
 - ☒ Documentation complete
 - ☒ All endpoints tested
-

Day 30: Monitoring, Deployment & Final Testing

Time: 7-8 hours

Status: ☐ Not Started

Tasks:

- ☐ **30.1** Create monitoring dashboard in `monitoring/dashboard.py`

```

import streamlit as st

st.title("Educational RAG Monitoring")

# Metrics
col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Queries", get_total_queries())
col2.metric("Avg Latency", f"{get_avg_latency():.2f}s")
col3.metric("Cache Hit Rate", f"{get_cache_hit_rate():.1%}")
col4.metric("Avg Confidence", f"{get_avg_confidence():.2f}")

# Charts
st.line_chart(get_queries_over_time())
st.bar_chart(get_top_topics())

# Recent queries
st.dataframe(get_recent_queries())

```

☐ 30.2 Set up logging aggregation

```

from loguru import logger

logger.add(
    "logs/rag_{time}.log",
    rotation="500 MB",
    retention="10 days",
    level="INFO"
)

```

☐ 30.3 Create deployment configuration

- Dockerfile for containerization
- docker-compose.yml for full stack
- Environment variable documentation
- Deployment scripts

☐ 30.4 Create Dockerfile

```
FROM python:3.10-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

☐ **30.5** Run final integration tests

```
# Full system test
```

```
pytest tests/test_workflow.py -v
```

```
# Load testing
```

```
locust -f tests/load_test.py --headless -u 10 -r 2 -t 5m
```

☐ **30.6** Create deployment documentation

- Setup guide
- Configuration guide
- Troubleshooting guide
- API usage examples

☐ **30.7** Deploy to staging environment

```
docker-compose up -d
```

```
# Verify all services
```

```
docker-compose ps
```

```
curl http://localhost:8000/health
```

Acceptance Criteria:

- ☒ Monitoring dashboard running
 - ☒ Logging configured
 - ☒ Docker containers working
 - ☒ Full system deployed to staging
 - ☒ Load tests pass (10 concurrent users)
 - ☒ Documentation complete
-



Success Metrics

Technical Metrics

- ☐ Average query latency < 3 seconds
- ☐ Retrieval confidence > 0.75 (average)
- ☐ RAGAS faithfulness > 0.85
- ☐ RAGAS answer relevancy > 0.80
- ☐ Cache hit rate > 30%
- ☐ Hallucination score < 0.15
- ☐ Citation accuracy > 95%
- ☐ System uptime > 99%

Quality Metrics

- ☐ 90%+ of responses have valid citations
 - ☐ 85%+ of responses pass guardrails
 - ☐ Multi-turn conversations maintain context
 - ☐ Tool use accuracy > 95%
 - ☐ Code coverage > 80%
-



Deliverables Checklist

Code

- ☐ Complete source code in `src/`
- ☐ All tests in `tests/`
- ☐ API implementation in `api/`

- ☐ Scripts in `scripts/`
- ☐ Notebooks in `notebooks/`

Documentation

- ☐ README.md with setup instructions
- ☐ API documentation
- ☐ Architecture documentation
- ☐ Deployment guide
- ☐ Troubleshooting guide

Data

- ☐ Sample PDFs processed
- ☐ Sample transcripts processed
- ☐ Test dataset created
- ☐ Evaluation results

Infrastructure

- ☐ Docker containers configured
- ☐ Database schema created
- ☐ Pinecone index populated
- ☐ Redis cache configured



Common Issues & Solutions

Week 1 Issues

- **Docker won't start:** Check Docker Desktop running, ports not in use
- **Pinecone connection fails:** Verify API key, check region
- **BGE model download slow:** Use faster internet or download separately

Week 2 Issues

- **PDF parsing fails:** Install poppler-utils for pdf2image
- **Chunking too slow:** Batch process, use multiprocessing
- **Pinecone upload errors:** Check dimension matches (1024), batch uploads

Week 3 Issues

- **Retrieval too slow:** Reduce top_k, optimize Pinecone queries
- **HyDE generates bad answers:** Adjust prompt, try different temperature
- **Reranking bottleneck:** Use smaller model or increase batch size

Week 4 Issues

- **Groq rate limits:** Implement retry with backoff, use caching
- **Tool execution fails:** Add error handling, timeout limits
- **Context too long:** Implement truncation, increase compression

Week 5 Issues

- **RAGAS evaluation slow:** Run async, sample subset of queries
- **Cache misses too high:** Lower similarity threshold, debug cache keys
- **PostgreSQL connection pool exhausted:** Increase pool size

Week 6 Issues

- **Docker build fails:** Check dependencies, clear cache
- **API slow under load:** Add connection pooling, increase workers
- **Memory leaks:** Profile with memory_profiler, fix unclosed connections

Post-Implementation (Week 7+)

Continuous Improvement

- ☐ Collect user feedback
- ☐ Analyze query logs for patterns
- ☐ Identify common failure cases
- ☐ Fine-tune retrieval parameters
- ☐ Improve prompts based on real usage

Advanced Features (Future)

- ☐ Multi-modal support (images in documents)
- ☐ Voice query support
- ☐ Real-time collaborative features

- ☐ Advanced analytics dashboard
- ☐ A/B testing framework
- ☐ Fine-tuned embedding model
- ☐ Custom reranking model

Scaling

- ☐ Horizontal scaling with load balancer
- ☐ Regional deployment
- ☐ CDN for static assets
- ☐ Database replication
- ☐ Microservices architecture

Support & Resources

Documentation

- LangChain: <https://python.langchain.com/docs/>
- LangGraph: <https://langchain-ai.github.io/langgraph/>
- Pinecone: <https://docs.pinecone.io/>
- RAGAS: <https://docs.ragas.io/>

Community

- LangChain Discord
- Stack Overflow
- GitHub Issues

Last Updated: [Date]

Version: 1.0

Status: Implementation Ready 